

INFORME TÉCNICO

Aplicación web con cuatro componentes y despliegue en Docker

Panel de Procesos — React + Vite

Autor: Isaac Geovanny Betún Vergara

Carrera: Tecnología Superior en Desarrollo de Software

Asignatura: [completar — p. ej. Programación Integradora]

Docente: [completar] NRC: [completar]

Fecha: [completar]

1. Introducción

El presente informe documenta el desarrollo y despliegue de una aplicación web denominada «Panel de Procesos», construida con la biblioteca React y la herramienta de compilación Vite. La aplicación integra cuatro componentes independientes, cada uno encargado de un proceso de cálculo diferente, y se despliega mediante contenedores Docker. Adicionalmente, se describe el proceso de despliegue en el entorno en línea KillerCoda, el cual clona el repositorio Git del proyecto para ejecutarlo.

El objetivo académico es demostrar la separación de responsabilidades en componentes, la validación de datos de entrada y la contenerización de una aplicación de una sola página (SPA) para su publicación.

2. Objetivos

2.1. Objetivo general

Desarrollar una aplicación web en React compuesta por cuatro componentes con procesos diferenciados y desplegarla mediante Docker.

2.2. Objetivos específicos

- Implementar cuatro componentes funcionales: operaciones básicas, cálculo de IMC, cálculos del IEES y conversor de temperatura.
- Aplicar validación de datos de entrada en cada proceso para evitar resultados inconsistentes.
- Contenerizar la aplicación con un Dockerfile de múltiples etapas que optimice el tamaño de la imagen final.
- Desplegar la aplicación en el entorno KillerCoda a partir del repositorio Git.

3. Herramientas y tecnologías

- React 18 y Vite 5: biblioteca de interfaz y empaquetador para el desarrollo del frontend.
- JavaScript (ES6+) y CSS3: lógica de los procesos y sistema de estilos.
- Docker: contenerización mediante un build multi-etapa (Node 20 para compilar y Nginx para servir).
- Nginx: servidor web que entrega los archivos estáticos en producción.
- Git y GitHub: control de versiones y alojamiento del repositorio.
- KillerCoda: entorno en línea con Docker para el despliegue.

4. Descripción de la aplicación

La aplicación presenta una barra de navegación superior que permite alternar entre los cuatro procesos. Cada proceso se implementa como un componente React independiente, con su propio estado y color de acento, y muestra el resultado en un formato de comprobante. La siguiente tabla resume cada componente:

#	Componente	Proceso que realiza
1	Operaciones básicas	Suma, resta, multiplicación y división de dos números, con validación de división para cero e historial de las operaciones realizadas.
2	Cálculo de IMC	Índice de masa corporal (peso / estatura ²), con clasificación según los rangos de la OMS y una barra visual del resultado.
3	Cálculos IESS	Rol de pagos mensual: aporte personal (9,45 %), aporte patronal (12,15 %), fondos de reserva (8,33 %), líquido a recibir y costo total para el empleador.
4	Conversor de temperatura	Conversión entre Celsius, Fahrenheit y Kelvin, con validación del cero absoluto (−273,15 °C).

Tabla 1. Componentes de la aplicación y su proceso.

4.1. Componente 1 — Operaciones básicas

Recibe dos números reales y aplica la operación seleccionada (+, −, ×, ÷). Valida que ambos campos contengan números y bloquea la división para cero, mostrando un mensaje de error. Conserva un historial con las últimas seis operaciones realizadas.

4.2. Componente 2 — Cálculo de IMC

Calcula el índice de masa corporal con la fórmula **IMC = peso (kg) / estatura² (m²)**. La estatura se ingresa en centímetros y se convierte a metros. El resultado se clasifica en bajo peso, peso normal, sobrepeso u obesidad según los rangos de referencia de la OMS, y se representa sobre una barra de colores con un marcador.

4.3. Componente 3 — Cálculos IESS

A partir del sueldo mensual, genera un rol de pagos con los aportes al Instituto Ecuatoriano de Seguridad Social para un trabajador del sector privado en relación de dependencia. Calcula el aporte personal (9,45 %), el aporte patronal (12,15 %, que incluye IECE y SECAP), los fondos de reserva mensualizados (8,33 %, opcionales), el líquido a recibir por el trabajador y el costo total mensual para el empleador. Los valores se muestran en dólares estadounidenses.

4.4. Componente 4 — Conversor de temperatura

Convierte un valor entre las escalas Celsius, Fahrenheit y Kelvin usando las fórmulas $^{\circ}\text{F} = ^{\circ}\text{C} \times 9/5 + 32$ y $\text{K} = ^{\circ}\text{C} + 273,15$. Valida que el valor no se encuentre por debajo del cero absoluto, ya que no correspondería a una temperatura físicamente posible.

5. Estructura del proyecto

El proyecto se organiza de la siguiente manera; cada componente reside en su propio archivo dentro de la carpeta src/components:

```

panel-procesos/
├── Dockerfile           → Build multi-etapa (Node 20 + Nginx)
├── docker-compose.yml   → Levanta el contenedor
├── nginx.conf           → Configuración de Nginx (SPA)
├── package.json         → Dependencias y scripts
└── index.html           → HTML raíz

```

```
└─ src/
  └─ App.jsx           → Navegación entre los 4 procesos
  └─ main.jsx          → Punto de entrada de React
  └─ index.css         → Sistema de estilos
  └─ components/
    └─ OperacionesBasicas.jsx
    └─ CalculoIMC.jsx
    └─ CalculosIESS.jsx
    └─ ConversorTemperatura.jsx
```

6. Capturas de la aplicación en funcionamiento

A continuación se presentan las capturas de cada componente en ejecución. Se recomienda mostrar un cálculo de ejemplo con su resultado en cada uno.

 Inserta aquí la captura del componente «Operaciones básicas» con una operación resuelta.

Figura 1. Componente de operaciones básicas.

 Inserta aquí la captura del componente «Cálculo de IMC» mostrando la categoría y la barra.

Figura 2. Componente de cálculo de IMC.

 Inserta aquí la captura del componente «Cálculos IEES» con el rol de pagos generado.

Figura 3. Componente de cálculos del IEES.

 Inserta aquí la captura del componente «Conversor de temperatura» con las equivalencias.

Figura 4. Componente conversor de temperatura.

7. Despliegue en Docker

La aplicación se conteneriza con un Dockerfile de dos etapas. En la primera etapa, una imagen de Node 20 instala las dependencias y ejecuta la compilación de producción (npm run build), que genera los archivos estáticos en la carpeta dist. En la segunda etapa, una imagen de Nginx copia únicamente esos archivos y los sirve. De este modo, la imagen final no contiene Node ni las dependencias de desarrollo, y pesa aproximadamente 50 MB.

La configuración de Nginx aplica la regla try_files para redirigir cualquier ruta hacia index.html, lo que permite que la aplicación de una sola página funcione correctamente al recargar la página.

7.1. Construcción y ejecución local

```
docker build -t panel-procesos:1.0 .
docker run -d --name panel-procesos -p 8080:80 panel-procesos:1.0
```

Una vez ejecutado, la aplicación queda disponible en <http://localhost:8080>. También puede levantarse con un solo comando mediante docker compose up -d --build.

 Inserta aquí la captura de la ejecución de «docker build» y «docker run» en la terminal.

Figura 5. Construcción y ejecución del contenedor.

8. Despliegue en KillerCoda

El despliegue en el entorno en línea KillerCoda se realiza clonando el repositorio Git del proyecto, sin necesidad de subir archivos comprimidos. Los pasos son los siguientes:

Paso 1. Ingresar a un escenario con Docker (Docker Playground de KillerCoda).

Paso 2. Clonar el repositorio y ubicarse en la carpeta del proyecto:

```
git clone https://github.com/USUARIO/panel-procesos.git
cd panel-procesos
```

Paso 3. Construir la imagen y ejecutar el contenedor en el puerto 80:

```
docker build -t panel-procesos:1.0 .
docker run -d --name panel-procesos -p 80:80 panel-procesos:1.0
```

Paso 4. Verificar el contenedor con «docker ps» y abrir la aplicación mediante el Traffic Port Accessor de KillerCoda, indicando el puerto 80.

 Inserta aquí la captura de la clonación del repositorio (git clone) en KillerCoda.

Figura 6. Clonación del repositorio en KillerCoda.

 Inserta aquí la captura de «docker ps» mostrando el contenedor en ejecución.

Figura 7. Contenedor en ejecución.

 Inserta aquí la captura de la aplicación abierta desde el puerto expuesto en KillerCoda.

Figura 8. Aplicación desplegada y accesible desde el navegador.

9. Repositorio

El código fuente completo del proyecto se encuentra alojado en el siguiente repositorio Git:

<https://github.com/USUARIO/panel-procesos>

(Reemplaza la URL anterior por la de tu repositorio real antes de entregar el informe.)

10. Conclusiones

- Se desarrolló una aplicación web en React compuesta por cuatro componentes independientes, cada uno con un proceso de cálculo y validación de datos propio.
- El uso de un Dockerfile multi-etapa permitió obtener una imagen de producción ligera, separando el entorno de compilación del de ejecución.
- El despliegue mediante KillerCoda a partir del repositorio Git demostró un flujo de publicación reproducible que no depende de archivos comprimidos.
- La organización del código en componentes facilita el mantenimiento y la incorporación de nuevos procesos en el futuro.